

Festival

Nayra está en un festival jugando un juego en el cual el gran premio es un viaje a la Laguna Colorada. El juego consiste en usar fichas para poder comprar cupones. Comprar un cupón puede resultar en fichas adicionales. El objetivo es conseguir tantos cupones como sea posible.

Ella empieza el juego con A fichas. Hay N cupones, enumerados de 0 a $N - 1$. Nayra tiene que pagar $P[i]$ fichas ($0 \leq i < N$) para poder comprar el cupón i (y debe tener al menos $P[i]$ fichas antes de la compra). Nayra puede comprar cada cupón una vez como máximo.

Además, a cada cupón i ($0 \leq i < N$) se le asigna un **tipo**, denotado por $T[i]$, el cual es un numero entero **entre 1 y 4, inclusive**. Después de que Nayra compra el cupón i , el número de fichas restantes que tiene se multiplica por $T[i]$. Formalmente, si ella tiene X fichas en algún punto del juego, y compra el cupón i (lo cual requiere $X \geq P[i]$), entonces va a tener $(X - P[i]) \cdot T[i]$ fichas después de la compra.

Tu tarea es determinar cuáles cupones debe comprar y en qué orden debe hacerlo, para maximizar el número total de **cupones** que tiene al final. Si es que hay más de una secuencia de compras que logra hacer esto, puedes reportar cualquiera de ellas.

Detalles de implementación

Debes implementar la siguiente función:

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : el numero inicial de fichas que Nayra tiene.
- P : un arreglo de tamaño N especificando los precios de los cupones.
- T : un arreglo de tamaño N especificando los tipos de los cupones.
- Esta función es llamada exactamente una vez por cada caso de prueba.

La función debe retornar un arreglo R , el cual especifica las compras de Nayra de la siguiente forma:

- El tamaño de R debe ser el máximo número de cupones que puede comprar.
- Los elementos del arreglo son los índices de los cupones que debería comprar, en orden cronológico. Es decir, ella compra el cupón $R[0]$ primero, el cupón $R[1]$ como segundo, y así

sucesivamente.

- Todos los elementos de R deben ser únicos.

Si no se pueden comprar cupones, R debe ser un arreglo vacío.

Restricciones

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ para todo i tal que $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ para todo i tal que $0 \leq i < N$.

Subtareas

Subtarea	Puntaje	Restricciones adicionales
1	5	$T[i] = 1$ para todo i tal que $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ para todo i tal que $0 \leq i < N$.
3	12	$T[i] \leq 2$ para todo i tal que $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Nayra puede comprar todos los N cupones (en algún orden).
6	16	$(A - P[i]) \cdot T[i] < A$ para todo i tal que $0 \leq i < N$.
7	18	Sin restricciones adicionales.

Ejemplos

Ejemplo 1

Considera la siguiente llamada a la función.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Inicialmente, Nayra tiene $A = 13$ fichas. Puede comprar 3 cupones en el orden mostrado debajo:

Cupón comprado	Precio del cupón	Tipo del cupón	Numero de fichas luego de la compra
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

En este ejemplo, no es posible que Nayra compre más de 3 cupones, y la secuencia de compras descrita arriba es la única forma en la que puede comprar 3 cupones. Por lo tanto, la función debe retornar $[2, 3, 0]$.

Ejemplo 2

Considera la siguiente llamada a la función.

```
max_coupons(9, [6, 5], [2, 3])
```

En este ejemplo, Nayra puede comprar ambos cupones en cualquier orden. Por lo tanto, el procedimiento debe retornar $[0, 1]$ o $[1, 0]$.

Ejemplo 3

Considera la siguiente llamada a la función:

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

En este ejemplo, Nayra tiene solo una ficha, lo cual no es suficiente para comprar ningún cupón. Por lo tanto, la función debe retornar $[]$ (un arreglo vacío).

Evaluador

Formato de Entrada:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Formato de Salida:

```
S
R[0] R[1] ... R[S-1]
```

Aquí, S es el tamaño del arreglo R retornado por `max_coupons`.